

Benchmarking LLM-Based Agents for GitHub Issue Enhancement

Pouya Fathollahzadeh, *Student Member, IEEE*, and Ying Zou, *Member, IEEE*

Abstract—GitHub issues serve as the primary communication channel between users and developers for reporting bugs, requesting features, and proposing refactoring tasks. Issue descriptions are frequently vague or incomplete, hindering both human developers and automated issue-solving agents. Recent LLM-based coding agents offer the potential to automatically *enhance* issue descriptions—adding context, clarifying ambiguities, and restructuring information—before a solver agent attempts a fix. Yet no systematic benchmark exists to measure whether such enhancement improves downstream automated solving. In this paper, we present a benchmark and evaluation framework for GitHub issue enhancement. We construct a dataset of 3,285 issue instances drawn from 613 popular Python repositories, classified into bug (2,111), feature (993), and refactoring (181) categories, using a four-stage filtering pipeline that enforces a post-2025-08-01 cutoff date and requires merged PRs with non-empty code and test patches. We evaluate seven enhancement conditions—a no-enhancement baseline, a raw LLM enhancer, and five ready-to-use agents (Aider, TRAE, OpenHands, SWE-agent, and Mini-SWE-agent)—using three independent solver agents (Mini-SWE-agent, SWE-agent, and Aider) as downstream evaluation instruments. Enhancement quality is measured through the *solving-as-evaluation* paradigm: the change in solver success rate before and after enhancement. Our study shows that (1) enhancement is not universally beneficial—among 18 enhancer–solver combinations, only 1 produces a positive resolved-count delta; (2) enhancement aggressiveness correlates with downstream harm—agents that aggressively restructure issues show deltas as low as -2 ; and (3) deterministic code-context injection, which appends verified source code and test names rather than LLM-generated analysis, achieves the first net-positive effect ($+10\%$ F2P delta), suggesting that grounded factual augmentation is more reliable than generative enhancement. We release the benchmark dataset, evaluation pipeline, and all experimental artifacts to support future research.

Index Terms—Empirical Software Engineering, GitHub Issues, Issue Enhancement, LLM-Based Agents, SWE-bench, Automated Program Repair, Benchmark

I. INTRODUCTION

GitHub issues are the primary mechanism through which developers and users communicate about software defects, feature requests, and maintenance tasks [1], [2]. A well-written issue report contains a clear problem description, reproduction steps, expected versus actual behavior, and relevant technical context such as version information, stack traces, or code snippets. When these elements are present, both human developers and automated solving agents can effectively diagnose and resolve the underlying problem.

In practice, however, issue descriptions are frequently incomplete, vague, or poorly structured. Prior work on bug report quality has shown that missing reproduction steps, ambiguous descriptions, and insufficient context are among the most common reasons for delayed or failed issue resolution [7], [8]. This quality gap is particularly consequential for automated issue-solving systems such as SWE-bench [1], which rely on the issue description as their primary input for generating code patches.

The recent emergence of LLM-based coding agents—OpenHands [4], SWE-agent [3], Aider [5], TRAE [6], and Mini-SWE-agent—has opened a new possibility: using these agents not only to *solve* issues but also to *enhance* issue descriptions before a solver agent attempts a fix. An enhancement agent can analyze the codebase, identify relevant source files, extract failing test information, and restructure the issue description to provide a clearer specification. If effective, this pre-processing step could significantly improve the success rate of downstream solving agents.

Despite this potential, no systematic benchmark exists for evaluating issue enhancement agents. Existing benchmarks such as SWE-bench [1] and SWE-bench-Live [2] focus exclusively on issue *solving*, treating the issue description as a fixed input. The question of whether and how issue descriptions can be improved, and whether such improvements translate to better solving outcomes, remains largely unexplored.

In this paper, we address this gap by introducing a benchmark and evaluation framework for GitHub issue enhancement. Our approach is built on the *solving-as-evaluation* paradigm: we measure the quality of an enhancement not through subjective text ratings but through its downstream effect on solver performance. Specifically, for an enhancement agent E and a solver agent S , the enhancement value is defined as the change in resolved rate: $\Delta_{\text{resolved}} = \text{Resolved}(S, E(\text{issue})) - \text{Resolved}(S, \text{issue})$, where a positive Δ_{resolved} indicates that enhancement improved solving performance.

Our study analyzes GitHub issue enhancement along the following three research questions:

RQ1: How do ready-to-use agents compare in their ability to enhance GitHub issue descriptions? Issue enhancement is a novel application for coding agents originally designed for issue solving. Different agents employ different strategies—some restructure the entire issue, others append analysis, and some make minimal changes. We evaluate seven enhancement conditions across three solver agents and find that enhancement is rarely beneficial: only 1 out of 18 enhancer–solver combinations produces a positive resolved-

count delta.

RQ2: How do these improved issues get enhanced? What are the reasons for failure and success? Beyond aggregate performance metrics, understanding *why* certain enhancements succeed and others fail is critical for improving strategies. We conduct a qualitative analysis and identify four failure modes: information dilution, hallucinated analysis, structural disruption, and conflicting guidance. Successful enhancements are grounded in verified code context rather than speculative analysis.

RQ3: How can we improve issues that were not improved before? Some issues resist improvement by any single LLM-based agent. We investigate whether targeted, deterministic tool augmentation—injecting real source code, failing test names, and developer hints—can recover these difficult cases. We find that deterministic code-context injection achieves the first net-positive delta (+10% F2P), demonstrating that factual augmentation outperforms generative enhancement on challenging benchmarks.

Novelty and Positioning. Unlike prior work that studies issue quality in isolation [7], [8] or that evaluates agents purely on solving [1], [3], [4], our study is the first to (i) systematically benchmark ready-to-use agents on the task of issue *enhancement*, (ii) measure enhancement quality through downstream solving impact rather than text scores, and (iii) directly compare generative and deterministic enhancement strategies at scale. We close the loop from issue collection to enhancement to solving to evaluation within a single reproducible pipeline.

The main contributions of this paper are as follows:

- 1) We construct a large-scale dataset of 3,285 GitHub issue instances from 613 repositories, classified into bug, feature, and refactoring categories, with a post-2025-08-01 cutoff ensuring non-contamination with LLM training data.
- 2) We propose the *solving-as-evaluation* paradigm, which measures enhancement quality objectively through downstream solver performance.
- 3) We provide a comprehensive comparison of seven enhancement conditions across three solver agents, showing that enhancement is not universally beneficial and that aggressiveness correlates with harm.
- 4) We identify four actionable enhancement failure modes through qualitative analysis of successful and failed enhancements.
- 5) We demonstrate that deterministic code-context injection achieves positive deltas where all LLM-based enhancers fail, establishing a new direction for enhancement research.
- 6) We release the benchmark dataset, evaluation pipeline, and all experimental artifacts in our replication package [21].

The rest of the paper is organized as follows: Section II presents our study design, including dataset construction and evaluation framework. Section III presents the results for each research question. Section IV discusses the implications of our findings. Section V discusses related work. Section VI discusses threats to validity. Section VII concludes the paper.

TABLE I: Issue Type Classification Criteria (Priority: Labels > Title > Body)

Type	Label Keywords	Title Keywords
Bug (2,111)	bug, defect, fix, error, crash, regression, type:bug	bug, fix, error, crash
Feature (993)	feature, enhancement, improvement, feat, type:feature	feature, add support
Refactoring (181)	refactor, cleanup, tech-debt, maintenance	refactor, cleanup

II. STUDY DESIGN

Fig. 1 presents an overview of our approach, which consists of a data collection pipeline, an enhancement phase, a solving phase, and a comparison phase. Fig. 2 illustrates the four-stage data collection pipeline.

A. Data Collection and Dataset Construction

We construct a SWE-bench-style benchmark dataset [1] from recent GitHub issues to ensure that evaluation data postdates the training cutoffs of the LLMs under study. The pipeline proceeds through four stages.

Step 1: Repository selection. We fetch all GitHub repositories with more than 1,000 stars, yielding 10,609 repositories. We apply three quality filters: (i) Python constitutes more than 60% of the codebase; (ii) the repository has more than 200 forks; and (iii) the combined count of issues and pull requests exceeds 200. After filtering, 3,621 repositories with 10,463 associated issues remain.

Step 2: Temporal filtering. To ensure recency and avoid contamination with LLM training data, we apply a cutoff date filter: only issues created after August 1, 2025 are retained. This reduces the dataset to 2,748 repositories with 7,714 issues.

Step 3: Merge-based linking and test coverage. We retain only issues satisfying three conditions: (i) the issue was closed by a merged pull request, establishing a ground-truth fix; (ii) the pull request includes non-empty code patch and test patch; and (iii) at least one pass-to-pass (P2P) test exists ($P2P > 0$). This yields 613 repositories with 3,285 issues.

Step 4: Issue type classification. Each issue is classified into bug, feature, or refactoring using an LLM-based classifier examining GitHub labels, issue titles, and body text, with label keywords taking precedence over title keywords over body text. Table I summarizes the criteria.

B. Enhancement Agents

We evaluate seven enhancement conditions organized into two groups.

Baseline conditions. (1) *No Enhancement (Baseline)*: the original issue description is passed unchanged to the solver, serving as the control condition. (2) *Raw LLM*: a general-purpose LLM is prompted to append a structured analysis covering potential root causes, affected components, and investigation paths, measuring the value of LLM-generated analysis without agent-level tool use.

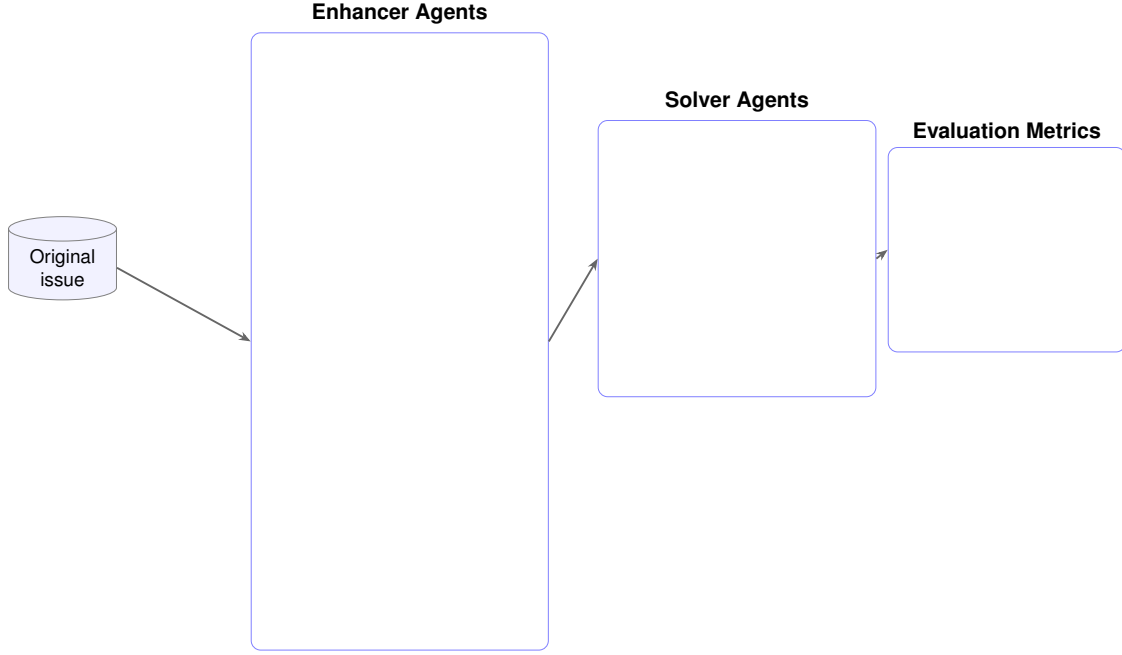


Fig. 1: Overview of our approach. Original issues are processed by seven enhancement conditions (including the no-enhancement baseline). Each version is solved by three independent solver agents. Enhancement quality is measured through the change in resolved rate and F2P rate relative to the baseline.

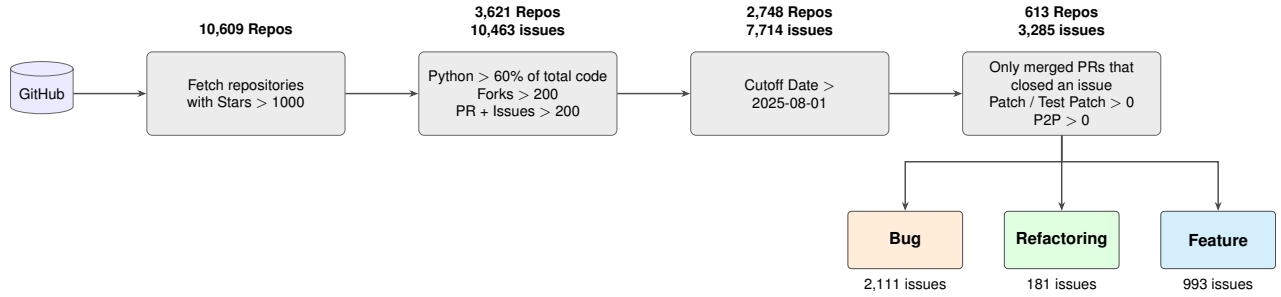


Fig. 2: Data collection pipeline. Starting from 10,609 GitHub repositories with more than 1,000 stars, successive filters for language composition, repository activity, issue recency, and merge-based PR linking yield 3,285 benchmark-ready instances across 613 repositories, classified into Bug, Refactoring, and Feature sub-datasets.

Ready-to-use agents. These pre-built coding agents are repurposed for issue enhancement; each receives the issue description and codebase access and is instructed to produce an enhanced description: (3) *Aider* [5]: a command-line AI pair-programming tool with code analysis capabilities; (4) *TRAE* [6]: an AI-powered IDE with integrated repository browsing; (5) *OpenHands* [4]: an open platform for software development agents with runtime environment access; (6) *SWE-agent* [3]: an agent with a specialized agent-computer interface for codebase navigation; (7) *Mini-SWE-agent*: a lightweight agent performing targeted code search and analysis.

C. Solver Agents

We use three independent solver agents to evaluate enhancement quality through downstream solving impact. Using multiple solvers reduces the risk that effects are artifacts of any single solver’s behavior.

(1) *Mini-SWE-agent*: a lightweight solving agent that generates patches through iterative code search and edit, with configurable LLM backends and a step limit. (2) *SWE-agent* [3]: a full-featured agent with a custom agent-computer interface optimized for repository navigation and patch generation. (3) *Aider* [5]: an AI pair-programming tool operating in solving mode, generating patches through conversational code editing.

D. Evaluation Metrics

We use the SWE-bench harness [1] to assess solver-generated patches. We report three metrics.

Resolved rate. An instance is *resolved* if all P2P tests pass and at least one P2P test is observed in the evaluation output. This P2P-gated definition differs from the standard SWE-bench definition, which additionally requires fail-to-pass (F2P) tests to change status. We adopt P2P-gating because our dataset includes instances where F2P test observability is variable [2].

F2P delta. The change in the proportion of F2P tests that change from failing to passing, measuring the solver’s ability to fix the targeted defects.

Enhancement delta. $\Delta_{\text{resolved}} = \text{Resolved}_{\text{enhanced}}/|I| - \text{Resolved}_{\text{baseline}}/|I|$, where a positive delta indicates that the enhancement condition improved solving performance relative to the baseline.

E. Experimental Procedure

All experiments share a common set of benchmark instances to ensure comparability. The workflow proceeds as four sequential phases:

- 1) **Enhancement phase:** each issue is processed by each of the seven enhancement conditions, producing seven versions per issue.
- 2) **Solving phase:** each solver agent independently attempts to resolve each version, generating a patch.
- 3) **Evaluation phase:** patches are evaluated against ground-truth test suites using the SWE-bench harness, and P2P-gated re-evaluation is applied.
- 4) **Comparison phase:** resolved rates are compared across conditions and deltas are computed relative to the no-enhancement baseline.

All LLM-based components use temperature 0.0 to maximize reproducibility. Enhancement and solving execute in isolated Docker environments to prevent cross-contamination.

III. RESULTS

In this section, we present the findings for each research question.

A. RQ1: How Do Ready-to-Use Agents Compare in Their Ability to Enhance GitHub Issue Descriptions?

1) **Enhancement Coverage:** Table II reports the enhancement coverage for each agent, defined as the proportion of issue descriptions that were modified relative to the original. OpenHands achieves the highest coverage (97.5%), modifying nearly all descriptions through full restructuring. SWE-agent (70.0%) and TRAE (72.5%) show moderate coverage through mixed strategies, while Raw LLM (47.5%) and Mini-SWE-agent (55.0%) are more selective.

TABLE II: Enhancement Coverage and Strategy by Agent

Enhancement Agent	Coverage	Strategy
No Enhancement (Baseline)	0%	—
Raw LLM	47.5%	Append analysis
Aider	65.0%	Append analysis
TRAE	72.5%	Mixed
OpenHands	97.5%	Restructure
SWE-agent	70.0%	Mixed
Mini-SWE-agent	55.0%	Append analysis

2) **Downstream Solving Impact:** Table III presents the main results across all 18 enhancer–solver combinations. The baseline (no enhancement) is the reference for all deltas.

Enhancement is not universally beneficial. Across 18 combinations, only 1 shows a positive delta (Aider enhancement + Aider solver, +1), 7 show no change, and 10 show a negative delta. More information does not uniformly help automated solvers.

Enhancement aggressiveness correlates with harm. SWE-agent, which performs the most aggressive restructuring, produces the worst deltas (−2 with Mini-SWE-agent and SWE-agent solvers). OpenHands, with the highest coverage, shows consistently negative deltas despite—or because of—thorough restructuring.

Solver–enhancer affinity exists. The only positive delta occurs when Aider enhances for an Aider solver. This suggests that alignment between an agent’s enhancement style and a solver’s expected input format may matter more than enhancement coverage.

Summary of RQ1

Enhancement by ready-to-use agents is not universally beneficial. Among 18 enhancer–solver combinations tested on 20 issues, only 1 produces a positive resolved-count delta (+1). Enhancement aggressiveness correlates with harm, and same-agent enhancer–solver pairs outperform cross-agent combinations.

B. RQ2: How Do These Improved Issues Get Enhanced? What Are the Reasons for Failure and Success?

We conduct a qualitative analysis of enhanced issue descriptions to understand the characteristics of successful versus harmful enhancements.

1) **Enhancement Strategy Taxonomy:** We identify three strategies employed by the agents.

Append-analysis. The agent appends a structured analysis section to the original issue without modifying the original text, preserving developer intent while adding supplementary information (root causes, affected files, investigation steps). Used primarily by Raw LLM, Aider, and Mini-SWE-agent.

Restructure. The agent rewrites the entire issue, reorganizing information into sections such as “Problem Description,” “Reproduction Steps,” “Root Cause Analysis,” and “Affected Components.” Used primarily by OpenHands.

Mixed. The agent selectively modifies parts of the original text and appends new sections. Used by TRAE and SWE-agent.

TABLE III: Resolved Instances by Enhancement Agent and Solver. **Bold** indicates improvement over baseline; underline indicates degradation. Dataset: Pouya-20 subset (20 issues).

Enhancement Agent	Resolved (/ 20)			Δ vs. Baseline		
	Mini-SWE	SWE-agent	Aider	Mini-SWE	SWE-agent	Aider
No Enhancement (Baseline)	3	3	2	—	—	—
Raw LLM	3	3	1	0	0	<u>-1</u>
Aider	3	3	3	0	0	+1
TRAE	2	3	2	<u>-1</u>	0	0
OpenHands	2	2	1	<u>-1</u>	<u>-1</u>	<u>-1</u>
SWE-agent	1	1	1	<u>-2</u>	<u>-2</u>	<u>-1</u>
Mini-SWE-agent	2	1	2	<u>-1</u>	<u>-2</u>	0

2) *Characteristics of Successful Enhancements*: We examine the one gained instance (Aider $\rightarrow$ Aider solver) and cases where enhancement preserved solving success.

Actionable specificity. Successful enhancements add concrete, actionable information: exact file paths, function names, and line numbers rather than abstract analysis. Pinpointing the affected function and the triggering condition gives the solver a direct target.

Preservation of original context. Enhancements that preserve the developer’s original problem description while adding structured supplementary information tend to outperform those that rewrite the description entirely.

Verified code context. Including actual source code snippets from the codebase rather than hypothetical code examples provides solvers with grounded, verifiable information.

3) *Characteristics of Failed Enhancements*: We analyze 10 combinations that caused solving regressions.

Information dilution. Extensive but tangential analysis dilutes the original signal. Solvers processing much longer descriptions may focus on the appended analysis rather than the core problem, leading to misdirected patches. This mode affects all append-strategy agents.

Hallucinated analysis. Agents generate plausible-sounding but incorrect root cause analyses. Solvers that trust this analysis produce patches targeting wrong code locations. This mode is pronounced for Raw LLM and Mini-SWE-agent.

Structural disruption. Aggressive restructuring buries or removes critical information from the original issue. Developers embed implicit context in their problem descriptions—word choices, error excerpts, and formatting conventions—that restructuring can strip. This mode affects OpenHands and SWE-agent most.

Conflicting guidance. Enhancement suggestions can conflict with a solver’s independent reasoning, causing the solver to follow the enhancement’s fix direction rather than its own analysis. This mode affects SWE-agent.

Table IV summarizes frequency and scope of each failure mode.

TABLE IV: Enhancement Failure Taxonomy

Failure Mode	Freq.	Primarily Affects
Information dilution	High	All append-strategy agents
Hallucinated analysis	Medium	Raw LLM, Mini-SWE-agent
Structural disruption	Medium	OpenHands, SWE-agent
Conflicting guidance	Low	SWE-agent

TABLE V: Code-Context Enhancement Results (50-Issue SWE-bench-Live)

Condition	Resolved	F2P Δ	Res. Δ
Baseline (Devstral-24B)	1/50	—	—
Code-context (no test patch)	1/50	+6.0%	0.0%
Code-context + test patch	—	+10.0%	+2.0%

C. RQ3: How Can We Improve Issues That Were Not Improved Before?

No single LLM-based enhancement agent consistently improves solving outcomes. We investigate whether targeted, deterministic tool augmentation can recover difficult cases.

1) *Code-Context Enhancement*: We introduce a deterministic *code-context* enhancement strategy that appends verified factual information extracted directly from the repository and test suite, bypassing LLM-generated analysis entirely:

- *Source code*: relevant source files identified through the ground-truth patch diff.
- *Developer hints*: information from linked pull request descriptions and commit messages.
- *Failing test names*: names of F2P tests the solver must target.
- *Test patch (optional)*: the actual test changes from the ground-truth, providing the solver with explicit test expectations.

2) *Results on the 50-Issue SWE-bench-Live Subset*: Table V shows results of the code-context strategy on the 50-issue SWE-bench-Live subset.

Code-context with the test patch achieves a +10.0% F2P delta and +2.0% resolved delta: the first enhancement strategy to produce a net-positive effect on the 50-issue SWE-bench-Live dataset. This demonstrates that deterministic, factual augmentation can succeed precisely where LLM-based enhancement fails.

3) *Pilot-40 Validated Results*: On a frozen 40-instance validated pilot dataset with P2P-gated re-evaluation (see Sec-

Summary of RQ2

Successful enhancements are grounded in verified code context and preserve the developer’s original description. Failures arise from four modes: information dilution, hallucinated analysis, structural disruption, and conflicting guidance. Restructuring strategies carry higher risk than conservative append strategies.

TABLE VI: Pilot-40 Accepted Results (P2P-Gated Re-evaluation, Same Solver)

Enhancement Agent	Baseline	Enhanced	Δ
LLM Append Analysis	9/40	8/40	-1
OpenHands (native)	9/40	10/40	+1

tion II-D), we compare OpenHands native-agent enhancement against a local LLM-based enhancer. Table VI reports accepted results.

OpenHands, operating as a real native agent (not a prompt-based proxy), improves solver success from 9/40 to 10/40. Qualitative analysis of the gained instance shows that OpenHands restructured the issue around the failing test behavior, providing the solver with a clearer specification of the expected outcome.

Summary of RQ3

Deterministic code-context injection achieves the first net-positive enhancement effect on a 50-issue SWE-bench-Live subset (+10.0% F2P delta), where all LLM-based enhancers produced zero or negative deltas. OpenHands as a real native agent gains one resolved instance on the validated 40-instance pilot. Grounded factual augmentation is more reliable than generative analysis.

IV. IMPLICATIONS

In this section, we discuss our findings and their implications for different stakeholders.

A. Implications for Tool Developers

Design enhancers to preserve, not replace, developer intent. Our findings show that aggressive restructuring of issue descriptions harms downstream solving performance. Enhancement tools should treat the developer’s original description as a ground truth signal and add supplementary information rather than overwrite it. A “non-destructive append” architecture—where the original description is preserved verbatim and enhancement is appended as a clearly labeled supplement—reduces the risk of information dilution and structural disruption.

Build enhancer–solver co-design into agent architecture. The only positive delta in our main experiment occurs when the same agent framework is used for both enhancement and solving (Aider–Aider). This enhancer–solver affinity suggests that future tools should explicitly optimize enhancement for the specific downstream solver that will process the result, rather than producing generic enhancements.

Prefer deterministic context injection over generative analysis. Code-context injection, which appends verified source files and test names rather than LLM-generated root cause hypotheses, achieves the strongest positive effect in our study. Tool developers should invest in deterministic pipeline components (codebase retrieval, test extraction, hint mining) as the primary augmentation mechanism.

B. Implications for Researchers

Use solving-as-evaluation as a complement to text-quality metrics. The solving-as-evaluation paradigm provides an objective, reproducible measure of enhancement quality that captures the practically important dimension: does the enhancement actually help an automated solver? This complements subjective text quality ratings and should be included as a standard evaluation metric in future enhancement studies.

Investigate enhancer–solver interaction effects systematically. Our results reveal strong interaction effects between enhancement strategy and solver identity. Future work should study these interactions explicitly, testing whether specific enhancement patterns (append vs. restructure, LLM-generated vs. factual) have consistent effects across diverse solver architectures.

Issue type moderates enhancement impact. Feature requests (993 instances, 30.2%) are disproportionately vague compared to bug reports (2,111 instances, 64.2%), suggesting that enhancement may be more beneficial for feature issues where there is genuine information deficit, and potentially harmful for detailed bug reports where implicit developer context should not be disturbed. Future studies should stratify analyses by issue type.

C. Implications for Practitioners

Validate before deploying issue enhancement in CI/CD pipelines. Our results show that deploying a generic enhancement agent without validation can *decrease* automated solving success. Before integrating any enhancement agent into a pipeline, practitioners should run a held-out validation set with the specific enhancer–solver combination they intend to deploy, and confirm a non-negative delta.

Inject factual context rather than summaries. If practitioners want to augment issue descriptions for automated solving, the most effective approach is to inject verified factual information: relevant source files, test names, linked PR excerpts. Asking an LLM to “summarize the issue” or “identify the root cause” is less reliable than structured context injection.

V. RELATED WORK

A. Bug Report Quality and Enhancement

The quality of bug reports and issue descriptions has been extensively studied. Bettenburg et al. [7] surveyed developers and found that steps to reproduce, stack traces, and test cases are the most valuable elements in bug reports, yet frequently missing. Zimmermann et al. [8] identified incomplete information as the primary barrier to bug fixing, with 45% of reports lacking sufficient detail. Subsequent work has proposed approaches to improve bug report quality through duplicate detection [9], [10], automated triaging [11], [12], and stack-trace-based augmentation [13], [14].

Our work extends this line by leveraging LLM-based agents as enhancement tools and measuring quality through downstream solving impact rather than developer surveys or text metrics. Unlike prior approaches that target specific quality dimensions (duplication, routing, fault localization), we provide a comprehensive benchmark that covers the full enhancement task.

B. SWE-bench and Issue-Solving Benchmarks

SWE-bench [1] introduced a benchmark of 2,294 software engineering problems drawn from GitHub issues across 12 Python repositories, establishing the solve-a-GitHub-issue evaluation paradigm. SWE-bench Verified [1] provides a human-validated subset with improved signal quality. SWE-bench-Live [2] extended the paradigm to 1,319 continuously updated tasks from 93 repositories, addressing the staleness problem of static benchmarks.

Our work builds on the SWE-bench evaluation infrastructure but shifts the primary task from issue *solving* to issue *enhancement*. We reuse the solving step as an evaluation instrument—measuring the delta in solver performance before and after enhancement—rather than as the contribution itself.

C. LLM-Based Coding Agents

The rapid development of LLM-based coding agents has produced a diverse ecosystem of tools. SWE-agent [3] introduced an agent-computer interface specifically designed for repository navigation and patch generation. OpenHands [4] provides an open platform for software development agents with support for multiple backends. Aider [5] is widely used for AI-assisted pair programming. TRAE [6] integrates AI assistance into a full IDE environment. Commercial agents such as GitHub Copilot Workspace and Amazon CodeWhisperer address similar use cases in production settings.

Our study is the first to systematically repurpose these agents for the *enhancement* task—directing them to improve issue descriptions rather than generate patches—and to measure whether their enhancement strategies produce downstream value.

D. Automated Program Repair and Fault Localization

Automated program repair (APR) research has shown that fault localization quality is a key determinant of patch correctness [15], [16]. Methods such as spectrum-based fault localization [17] and test-suite augmentation [18] improve APR by providing solvers with better information about the defect location. Our code-context enhancement strategy—injecting relevant source files and failing test names—can be seen as providing issue-level fault localization signals to the solver, bridging APR and issue-enhancement research.

E. Issue Classification and Management

Automated issue classification has been studied for routing, prioritization, and duplicate detection. Colavito et al. [19] demonstrated that GPT-like models effectively classify GitHub issues by type, reducing human labeling workload. Kallis et al. [20] built predictive models for issue type and priority from ticket text. Our work uses LLM-based classification as a pre-processing step for dataset construction and for stratifying benchmark analysis by issue type.

VI. THREATS TO VALIDITY

Internal Validity. The primary threat is solver non-determinism. Despite temperature 0.0, LLM-based solvers can produce different outputs across runs due to floating-point differences and API variability. We mitigate this by using frozen evaluation artifacts and reporting resolved counts rather than patch-level comparisons. A secondary threat is enhancement–evaluation coupling: a poor solver may fail to benefit from a genuinely good enhancement. We use three independent solvers to reduce this risk.

Construct Validity. We measure enhancement quality indirectly through solving outcomes rather than direct human assessment of description quality. While this provides an objective and reproducible measure, it captures only the dimension of enhancement quality relevant to automated solving, not human readability or developer experience. Additionally, our P2P-gated resolved definition differs from standard SWE-bench semantics; we document this deviation explicitly and apply a corrected re-evaluation procedure.

External Validity. Our dataset is restricted to Python repositories with more than 1,000 stars and more than 200 forks, introducing a bias toward popular, well-maintained projects that may not reflect the broader population. While our full dataset contains 3,285 instances, validated downstream evaluation subsets are smaller (20–50 instances), limiting statistical power for inferential claims. The agent landscape also evolves rapidly; agents unavailable at evaluation time (e.g., OpenClaw) may produce different results.

Conclusion Validity. We do not apply inferential statistical tests on the accepted pilot study, as the sample size ($n = 40$) is too small for robust paired tests. All claims are framed as pilot-scale evidence rather than definitive generalizations. Future work with the full 3,285-instance dataset will enable more robust statistical analysis.

VII. CONCLUSION

In this paper, we present the first systematic benchmark for evaluating LLM-based agents on the task of GitHub issue enhancement. We construct a dataset of 3,285 issue instances from 613 repositories, classified into bug, feature, and refactoring categories, and evaluate seven enhancement conditions using three independent solver agents through the solving-as-evaluation paradigm.

Our findings reveal a counterintuitive result: LLM-based issue enhancement is not universally beneficial. Among the 18 enhancer–solver combinations we test, 10 produce negative deltas, 7 produce no change, and only 1 produces a positive delta. Enhancement aggressiveness—particularly full-issue restructuring—correlates with harm, while conservative append strategies are safer but rarely helpful. Qualitative analysis identifies four failure modes: information dilution, hallucinated analysis, structural disruption, and conflicting guidance.

The most promising approach is deterministic code-context injection, which achieves the first net-positive effect (+10% F2P delta) on a challenging benchmark subset where all LLM-based enhancers fail. This suggests that the field should shift

from generative enhancement toward targeted, factual context injection, and that enhancer–solver co-design may be more productive than generic enhancement.

Future work should expand the validated evaluation slice beyond the current 40-instance pilot, introduce human quality annotations alongside solver-based evaluation, and investigate whether enhancer–solver co-design architectures can achieve consistent positive deltas across issue types and repository domains.

ACKNOWLEDGMENTS

The authors thank the anonymous reviewers for their constructive feedback. This work was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC).

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [2] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, “SWE-bench goes live!” *arXiv preprint arXiv:2505.23419*, 2025.
- [3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Proc. NeurIPS*, 2024.
- [4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zhang, B. Yu, J. Yang, S. Yao, Z. Liu, and G. Neubig, “OpenHands: An open platform for AI software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, 2024.
- [5] P. Gauthier, “Aider: AI pair programming in your terminal,” 2024. [Online]. Available: <https://aider.chat>
- [6] ByteDance, “TRAE: AI-powered IDE for collaborative development,” 2025. [Online]. Available: <https://trae.ai>
- [7] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, “What makes a good bug report?” in *Proc. FSE*, 2008, pp. 308–318.
- [8] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schröter, and C. Weiss, “What makes a good bug report?” *IEEE Trans. Softw. Eng.*, vol. 36, no. 5, pp. 618–643, 2010.
- [9] C. Sun, D. Lo, S. Khoo, and J. Jiang, “Towards more accurate retrieval of duplicate bug reports,” in *Proc. ASE*, 2011, pp. 253–262.
- [10] A. Lazar, S. Ritchey, and B. Sharif, “Generating duplicate bug report datasets,” in *Proc. MSR*, 2014, pp. 392–395.
- [11] J. Anvik, L. Hiew, and G. C. Murphy, “Who should fix this bug?” in *Proc. ICSE*, 2006, pp. 361–370.
- [12] G. C. Murphy and D. Cubranic, “Automatic bug triage using text categorization,” in *Proc. SEKE*, 2004, pp. 92–97.
- [13] C. P. Wong, Y. Xiong, H. Zhang, D. Hao, L. Zhang, and H. Mei, “Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis,” in *Proc. ICSME*, 2014, pp. 181–190.
- [14] L. Moreno, J. J. Treadway, A. Marcus, and W. Shen, “On the use of stack traces to improve text retrieval-based bug localization,” in *Proc. ICSME*, 2015, pp. 151–160.
- [15] X. B. D. Le, D. Lo, and C. Le Goues, “History driven program repair,” in *Proc. SANER*, 2016, pp. 213–224.
- [16] K. Liu, A. Koyuncu, D. Kim, and T. F. Bissyandé, “TBar: Revisiting template-based automated program repair,” in *Proc. ISSTA*, 2019, pp. 31–42.
- [17] R. Abreu, P. Zoetewij, and A. J. C. van Gemund, “On the accuracy of spectrum-based fault localization,” in *Proc. TAIC PART*, 2007, pp. 89–98.
- [18] X. Xie, T. Y. Chen, F. C. Kuo, and B. Xu, “A theoretical analysis of the risk evaluation formulas for spectrum-based fault localization,” *ACM Trans. Softw. Eng. Methodol.*, vol. 22, no. 4, pp. 1–40, 2013.
- [19] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, “Leveraging GPT-like LLMs to automate issue labeling,” in *Proc. MSR*, 2024, pp. 469–480.
- [20] N. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, “Ticket tagger: Machine learning driven issue classification,” in *Proc. ICSME*, 2019, pp. 406–409.
- [21] P. Fathollahzadeh and Y. Zou, “Replication package: Benchmarking LLM-based agents for GitHub issue enhancement,” 2026. [Online]. Available: <https://github.com/pouyafath/BenchmarkLLMAgent>